Classical Generative AI

L-Systems & Iterated Function Systems

Practical Worksheet — Guided Implementation

Deterministic L-Systems

Stochastic L-Systems

IFS Fractals

Session Roadmap

1 Rewriting Rules

How symbolic rewriting works — deterministic L-systems

2 Turtle Rendering

Interpreting symbol strings as geometry (F, +, -, [,])

3 Stochastic L-Systems

Markov-style probabilistic rule selection

4 IFS Fractals

Iterated affine transforms, burn-in, Barnsley fern

5 Exploration

Controlled experiments on parameters & diversity

6 Theory Questions

Production systems, Markov property, generativity

1 — How Rewriting Rules Work

The Idea

- Start with a string (the axiom)
- Apply substitution rules in parallel to every symbol
- Repeat for n iterations
- The string grows exponentially — encodes complex structure from simple rules

Demo trace:

```
A → AB      B → A

step 0: A
step 1: AB
step 2: ABA
step 3: ABAAB
```

Your Task

Implement two functions:

```
rewrite_once(sentence, rules)
  → replace each symbol

generate_lsystem(axiom, rules, n)
  → iterate n times
```

Hint:

```
rules.get(symbol, symbol)
```

Then test with the small rule set:

```
F → F+F-F-F+F   (angle 90°)
```

2 — Turtle Rendering

F	Draw forward by step
+	Turn left by angle
−	Turn right by angle
[	Push state (x, y, θ)
]	Pop state (return)

Main Challenge: Branching Plant

```
axiom: X
X → F+[[X]-X]-F[-FX]+X
F → FF
```

Parameters to choose:

- n_iter — try 4 or 5 first
- angle — around 20°–30° for plant look
- step — controls segment size

The [and] commands create branches — the stack saves and restores the turtle's position and heading.

3 — Stochastic L-Systems

Markov-Style Local Choices

- Each symbol can have multiple replacement rules, each with a probability
- At each rewrite step, sample which rule to apply
- Same grammar → different outputs each run
- Probabilities must sum ≈ 1.0

```
F → F[+F]F[-F]F   (p=0.34)
F → F[+F]F         (p=0.33)
F → F[-F]F         (p=0.33)
```

Implement 3 Functions

1. **`sample_rule(symbol, rules, rng)`**

   Cumulative probability sampling — draw u∈[0,1), walk through choices

2. **`rewrite_stochastic(sentence, rules, rng)`**

   Replace each symbol using sample_rule

3. **`generate_stochastic(axiom, rules, n, seed)`**

   Initialize RNG, iterate n times

4 — Iterated Function Systems

The Algorithm

- Start at point x = [0, 0]
- Randomly pick transform i with probability p_i
- Apply: $x \leftarrow A_i \cdot x + b_i$
- Repeat thousands of times → points converge to fractal attractor

Why Burn-in?

- First points are influenced by the arbitrary initial position
- Burn-in = discard initial transient points
- After burn-in, points lie on the stable attractor

Barnsley Fern

4 affine transforms with probabilities:

Transform	Role	Prob
f_1	Stem (contracts to base)	0.01
f_2	Main leaflet copy	0.85
f_3	Left branch	0.07
f_4	Right branch	0.07

Implement:

```
ifs_sample(transforms, probs,
           n_points, burnin, seed)
```

Use rng.choice for sampling, store points after burn-in

5 — Exploration Assignments

A Deterministic Scaling

- Run small rule set with n_iter ∈ {1, 2, 3, 4}
- Record sentence length + visual complexity
- Fix angle, then vary it — compare geometry

Loop over params, store in list of dicts, print table

B Stochastic Diversity

- Same rules, vary seed (≥ 5 values)
- Measure: sentence length, branch count [
- Change probability vector, compare variety

Write analyze_sentence() returning metrics dict

C IFS Convergence

- Vary burn-in ∈ {0, 20, 100, 500}
- Vary n_points ∈ {2k, 10k, 50k}
- Compare visual stability and noise in subplots

Use subplots, same seed for fair comparison

6 — Theory Questions

1. Why is deterministic rewriting a production-system style method?
2. Where exactly is the Markov-style step in stochastic L-systems?
3. Why is IFS generative even without neural networks?